Version: 15082023

Python Explorer

Overview

Python is essential for data analysts due to its simplicity, versatility, and vast ecosystem. It enables efficient data manipulation, analysis, and visualization. With Python, analysts can automate tasks, build machine learning models, and extract valuable insights from data.

In this course, you will learn the programming skills to perform data analysis and automate computer tasks, from basic programming syntax to writing algorithms, you will finish the course knowing how to transform data into actionable insights and using computational thinking for problem-solving.

Unit 1

Introduction

What is Python

- Most Popular language on GitHub
- An high-level and general-purpose programming language
- Python is an interpreted language - can run line by line
- Large community support, especially for data analysis and machine learning
- What can we do with Python programming?
    1. Task automation
    2. Data wrangling
    3. Sales Forecast
    4. Machine Learning

Where do we use Python

- Jupyter Notebook - a web-based IDE
- Google Colab - a web-based IDE
- PyCharm - a IDE designed for Python development
- Anaconda - a Python distribution tool bundled with a collection of Python packages and tools for data science

Print()

- In Python, the print() function is used to tell a computer to talk
- You can print a sentence in string: print("hello world")
- You can also print a number in integer: print(256)
- You can also print more than one value: print("hello world", 256)

Unit 2

Calculation & Variables

What will you learn

- The basic calculation operands, including integer division and modulo
- What is a variable
- How is variables associated with objects
- The sequence of code execution

Calculations with Python

Name	Operator
Addition	+
Subtraction	-
Multiplication	*
Division	/
Exponentiation	**
Modulo	%
Integer Division	//

Order of execution

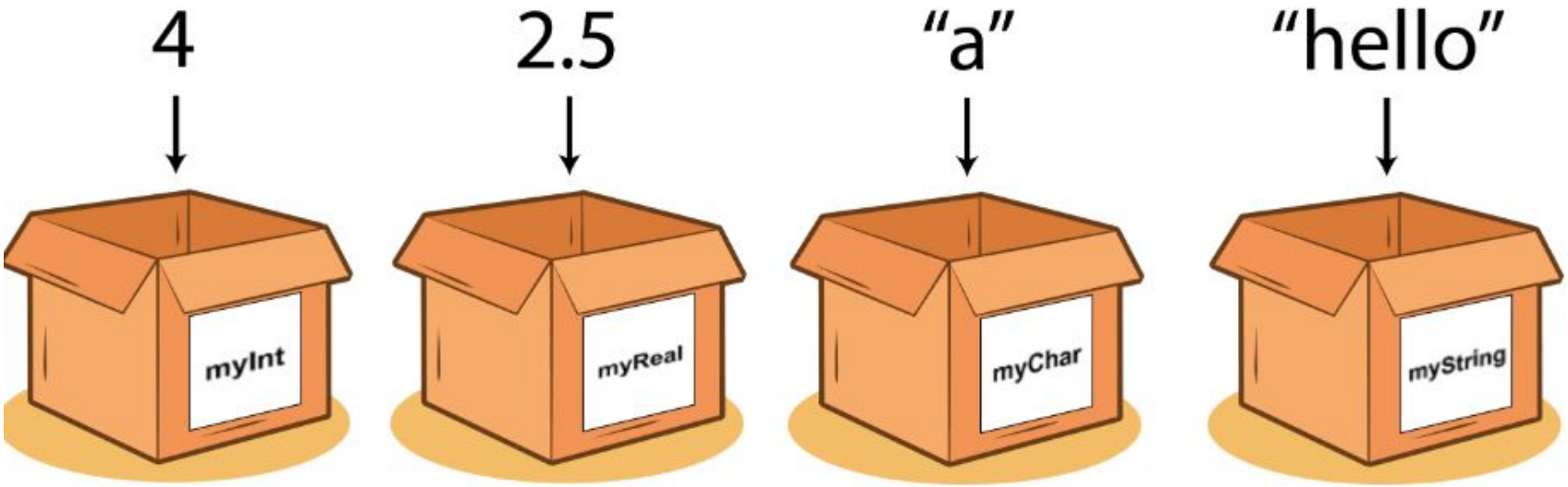
- Plus or minus: print(4 - 1 + 2)
- Multiplications and divisions are calculated before additions and subtractions: print(1 + 5 * 2 + 4)
- Brackets are evaluated first: print((1 + 5) * 2 + 4)
- Functions are calculated first, like using bracket: print(1 + max(2,5) + 4)
- Brackets can be nested: print(1 + max(2, (5 - 6)) + 4)

Numeric data type

- An **integer, or int**, is a whole number. It has no decimal point and contains all counting numbers (1, 2, 3, …) as well as their negative counterparts and the number 0.
- A **floating-point number, or a float**, is a decimal number. It can be used to represent fractional quantities as well as precise measurements.
- You can cast numbers in to int / float by using the corresponding constructor: **int() or float()**

What is Variable

- When there is a data we need to remember and reuse, we can create a variable which can store a value.
- We can image variables as labeled boxes
- Variables are pointers (a reference) to objects



Create & Update Variable

- In Python, we assign variables by using the equals sign (=), for example:
  - number = 123
  - text = "Hello World!"
  - aList = []
  - bList = aList

- We can update a variable using the = sign, for example:
  - num = 8
    print("Num:", num)
    num = num ** 2
    print("Updated Num:", num)

- Variables names cannot:
  - Contains space
  - Start with digits
  - Contains hyphen

The shorthand method

- Instead of writing:

```
number = 1
number = number + 2
number = number * 3
print(number) # Prints 9
```

- You can write:

```
number = 1
number += 2
number *= 3
print(number) # Prints 9
```

Unit 3

Data Types

What will you learn

- How the difference between data types affect programming
- The similarities between +,- and ==, <, >
- The difference between primitive types (numbers) and objects (eg. strings, lists)
- How to define a string, a list, a set and a dictionary

Boolean

- Only 2 possible values: True or False (Notice the first character **must** be in capital)
- Works well with the `AND`, `OR`, `NOT` operators
- Be careful not to enclose the boolean value with quotation marks
- A boolean expression is a statement that can either be True or False
- Any variable that is assigned one of the boolean type is called a boolean variable
- You can directly assign boolean values or boolean expression result to a variable

Relational Operators

Operator	Description
==	Equals
!=	Not Equals
>	Greater Than
>=	Greater Than or Equal
<	Less Than
<=	Less Than or Equal
in	In an iterable

String

- Computer programmers refer to blocks of text as strings. In Python a string is either surrounded by:
  - **single quotes**: 'Hello world’
  - **double quotes**: "Hello world"

- When Python sees the forward slash \, it interprets the following character(s) in the alternative way. In the quotation mark case, the escaped quotation mark \' is no longer seen as the end of string

- By using **three quote-marks** (""" or "') instead of one, we tell the program that the string doesn’t end until the next triple-quote.

- The **+ operator** can also add two strings. The process of combining two strings is called string concatenation. Performing string concatenation creates a brand new string comprised of the first string’s contents followed by the second string’s contents

- **in operation** checks if a string is part of another string: print("hell" in "hello")

- **len()** counts the number of characters. It works with many data types including string, list, dictionary and many other.

String formatting

- String formatting version 1: **The format function**

  Example of .format():

  ```
  print('I am {a}m tall. I have {b} arms, {c} hands and {d} fingers.'.format(
      a=height_in_meter,
      b=number_of_arms,
      c=number_of_hands,
      d=number_of_fingers))
  ```

- String formatting version 2: **f-String**

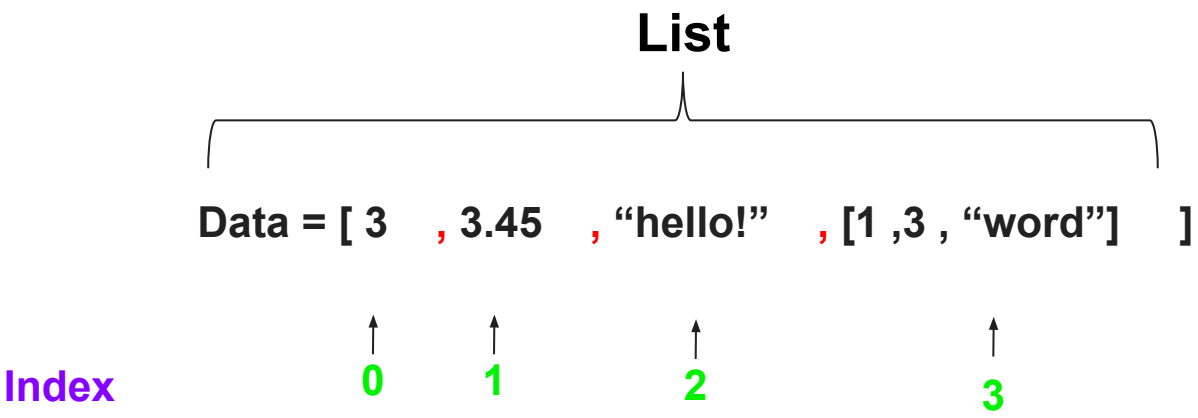
  Example of f-string:

  ```
  print(f'I am {height_in_meter}m tall. I have {number_of_arms} arms, {number_of_hands}
  hands and {number_of_fingers} fingers.')
  ```

String Manipulation

- **.lower()** returns the string with all lowercase characters
- **.upper()** returns the string with all uppercase characters
- **.title()** returns the string in title case, which means the first letter of each word is capitalized
- **.split()** is performed on a string, takes one optional argument, and returns a list of substrings found between the given argument
- **.join()** will join a list of strings together with a given delimiter
- **.strip()** takes one optional argument (default delimiter is space) and remove that character from the both ends of the string
- **.replace()** takes two arguments and replaces all instances of the first argument in a string with the second argument

List



- Enclose with square brackets **[** and **]**
- Unlike the other languages, it supports different data types within the same list.
- Orders are respected, start with zero indexing
- Lists are "mutable"

Working with List

- Example of creating a list:
  - some_list = [1, 2, 3, 4, 5]
  - another_list = [1, 'a', 2.0, False]
  - empty_list = []
- You can use **list()** to convert data into a list: number_list = list((1,2,3,4))
- Lookup in list with **in operator**: "mango" in ["apple", "orange"]
- Use **.index** to check the position of an item in a list: ["apple", "orange"].index('apple')
- Example of **selection in list**:
  - List[1] # item at position 1
  - List[[-1] # last item counting from the end
  - List[1:3] # items from position 1 (inclusive) to 3 (exclusive)
  - List[:2] # items from the beginning (inclusive) to position 2 (exclusive)

Working with List

- To change the value of a specific item, refer to the index number: **Syntax: List[index] = Value**
- **append()**: method to add an item to the end of the list
- **extend()**: method to append elements from another list to the current list
- **pop()**: method removes the specified item at a position

Dictionary

A dictionary is an unordered set of key: value pairs

Data = {“a”: 3 , “ b”:“hello!” , “ c”:[1 ,3 , “word”] }

Key : Value **Key : Value** **Key : Value**

- A dictionary begins and ends with braces { and }
- Each item consists of a key (e.g. "a") and a value (e.g. 3)
- Each key: value pair (e.g., "a": 3 or "b": ”hello!") is separated by a comma (,)
- Mutable (i.e. changeable) and orders are not respected

Create a Dictionary

```
person = {
            "name”  : "Klaus",
              "age”   : 18,
     "languages” : {"Programming":"python","human":"English"},
        "class"      :["Master","Junior"]
         }
```

- Keys can be strings or numbers
- Values can be any type, you can use a string, a number, a list, or even another dictionary
- You can create an empty dictionary and add key:value pair later: **Empty_dictionary = {}**

Get a Value with Key

- Once you have a dictionary, you can access the values in it by providing the key.

```
person = {
            "name”  : "Klaus",
              "age”   : 18,
      "languages” : {"Programming":"python","human":"English"},
          "class"      :["Master","Junior"]
          }
```

print(person[“name”])

Output:Klaus

- We can also use .get() function with the key on dictionary to access the corresponding value without key error: **inventory.get('orange')**

Adding or overwriting a key

- To add / overwrite a single key : value pair to a dictionary

```
inventory = {
    "apple": 2,
    "mango": 10,
    "pear": 3,
}
```

```
inventory['waterlemon'] = 10
print(inventory['waterlemon'])
```

Add new key value pair

```
inventory['apple'] = 4
print(inventory['apple'])
```

Overwrite key value pair

"in" and "not in" Operators

- The **"in"** and **"not in"** operators in Python are used to check if a value is present in a sequence or collection, such as a list, tuple, string, or set.

"in" Operator

```
fruits = ["apple", "banana", "orange"]
if "apple" in fruits:
    print("An apple is in the list.")
```

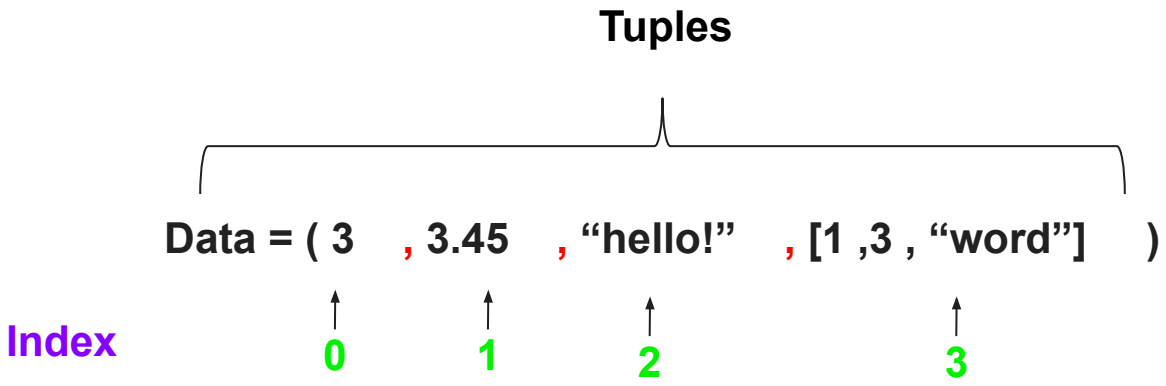
An apple is in the list.

“not in" Operator

```
fruits = ["apple", "banana", "orange"]
if ”grape" not in fruits:
    print("A grape is not in the list.")
```

A grape is not in the list.

Tuples



- Enclose with square brackets **(** and **)**
- It also supports different data types within the same list.
- Orders are respected, start with zero indexing
- Tuple are ”immutable"

Working with Tuples

- Tuple items are indexed and you can access them by referring to the index number: **Syntax: Tuple[index]**
- Similar to a list, but it does not have any functions.
- Tuple supports **auto-unpack**, for example:

```
tuple1 = 1, 2
item1, item2 = tuple1
print(item1, item2)
# Expected output: 1,2
```

Set

Set

Data = { 3 , 3.45 , “hello!” }

- Set is a collection of **unique items**. You may think it as either
  - Dictionary with no values
  - List with no duplicates
- You cannot access items in a set by referring to an index or a key

Working with Set

- You can use set() to **remove duplicate value of a list**, for example:

```
duplicate_list = [1,2,2,3,3,4]
unique_set = set(duplicate_list)
unique_list = list(unique_set)
print(unique_list)
# Expected output: [1,2,3,4]
```

None

- None is a very special datatype that indicates nothing is there. This data type has only one available value - None

- For example:
  - a = None
  - aList = [1,3,2]
    aList.sort()

Unit 4

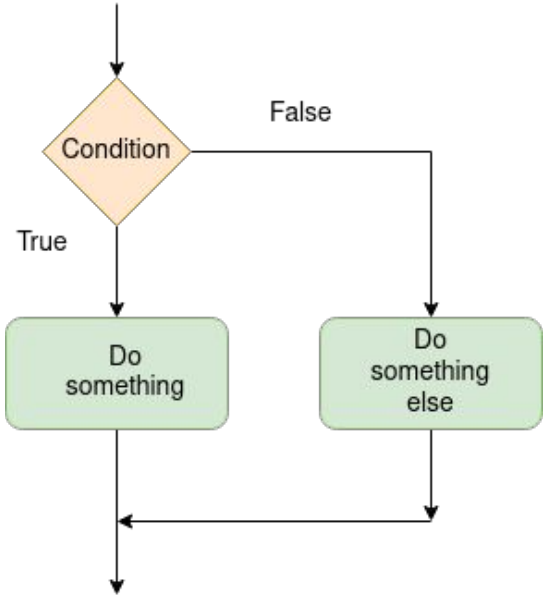
Control Flow

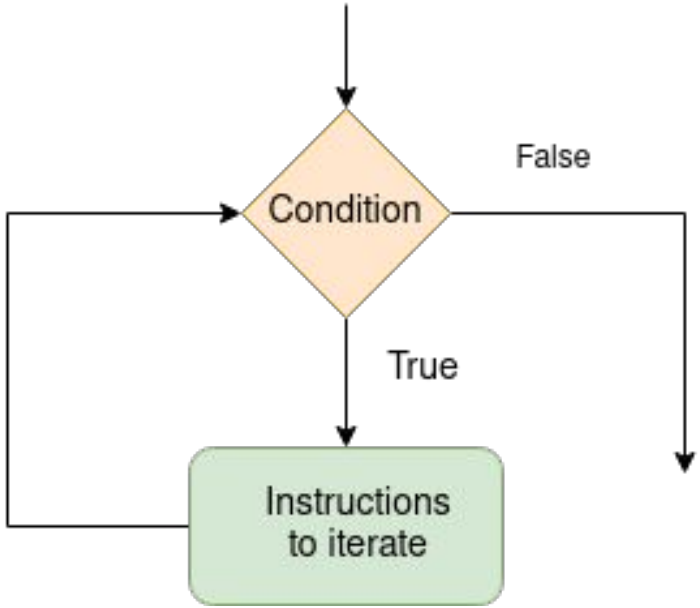
What will you learn

- How to control program execution with if-else conditional statements
- The Boolean operators: AND, OR and NOT
- How to control potential error with try-except statement

Control Flow

The Control Flow of a program is based on the **conditional statements** which tell the computer when it should execute certain blocks of code. If these conditions are met, then run this function.





If Statements

- The "if statement" is used to execute a block of code only if a given condition is true
- To write a if statement, you need:
  - The keyword if
  - The conditional statement
  - A colon :
  - An indented block of code

```
day_of_week = "Friday"

if day_of_week == "Friday":
  print("TGIF!")

# Expected output: TGIF!
```

```
day_of_week = "Sunday"

if day_of_week == "Friday":
  print("TGIF!")

# Expected output: Do nothing because day_of_week is not "Friday"
```

Indentation

- Indentation tells what is and what is not part of the if statement, for example:

```
if day_of_week == "Friday":
    code 1
    code 2
    code 3
```

- If we want code 3 outside of the if statement, we would have to unindent code 3:

```
if day_of_week == "Friday":
    code 1
    code 2
code 3
```

We usually use "TAB" key on the keyboard to indent the code.

Else Statements

The "else statement" in Python is used in conjunction with an if statement to execute a block of code when the condition of the if statement is false.

```
day_of_week = "Friday"

if day_of_week == "Friday":
  print("TGIF!")
else:
  print("Not Friday")

# Expected output: "TGIF!"
```

```
day_of_week = "Sunday"

if day_of_week == "Friday":
  print("TGIF!")
else:
  print("Not Friday")

# Expected output: "Not Friday"
because day_of_week is not
"Friday"
```

Else If Statements

The "elif statement" (short for "else if") allows you to check additional conditions after the initial if statement. It is used when you want to provide multiple alternatives.

```
day_of_week = "Friday"

if day_of_week == "Friday":
    print("TGIF!")
elif day_of_week in ["Saturday",
"Sunday"]:
    print("It is weekend!")
else:
    print("Need to work")

# Expected output: "TGIF!"
```

```
day_of_week = "Sunday"

if day_of_week == "Friday":
    print("TGIF!")
elif day_of_week in ["Saturday",
"Sunday"]:
    print("It is weekend!")
else:
    print("Need to work")

# Expected output: "It is weekend!"
```

Shorthand Assignment

We can assign different value to a variable based on a condition, for example:

```
age = 16
can_drink_alcohol = True if age >= 18 else False
can_drink_alcohol
```

Output:False

```
age = 23
can_drink_alcohol = True if age >= 18 else False
can_drink_alcohol
```

Output:True

Boolean Operators

- “and” combines two boolean expressions and evaluates as True if both its components are True, but False otherwise.
- “or” combines two expressions into a larger expression that is True if either component is True
- when “not” is applied to any boolean expression it reverses the boolean value. So if we have a True statement and apply a not operator we get a False statement

Try-except Statements

- We can use try-except statements to check for possible errors that a user might encounter, for example:

```
try:
    # some statement
except ErrorName:
    # do something when you see above error
```

```
try:
    number = float(input("Please provide a number"))
    print(100 / number)
except ZeroDivisionError:
    print("Input number should not be zero")
except Exception as e:
    print("Unexpected Error:", e)
```

- In above example:
  - If your input = 0, ZeroDivisionError will appear and will print "Input number should not be zero"
  - If your input = "name", other error message (stored in temporary variable e) will be printed

Try-except Statements

- The finally block is executed no matter the execution is successful or not, for example:

```
try:
    print("hello")
except:
    print("Some error happened")
finally:
    print("Finally")
```

- finally is powerful when break, continue, return is used to shortcut the current execution, for example:

```
for i in range(3):
    try:
        a = 1/0
    except:
        print("error in execution, break now")
        break # This break terminates the for loop.
    finally:
        print("finally - this is executed even if a break is used above")
```

Unit 5

Loops

What will you learn

- When should we use for loop and while loop
- The usage of built in range iterator
- How break and continue interrupt the loops

What is Loop

In Python, a loop is a programming structure that **allows you to repeatedly execute a block of code.** It's like a cycle that lets you do something multiple times without writing the same code over and over again. There are two main types of loops in Python:

For Loop

```
drinks = ['coffee', 'coke', 'juice']

for option in drink:
    print(option)
```

While Loop

```
While <condition>:
    action1
    action2
```

For-loop

- To write a for-loop, you need:
  - The keyword for <temp_variable> in <some_iterable>
  - A colon :
  - An indented block of code

- A temporary variable is a variable with short lifetime, usually to hold data that will soon be discarded, or before it can be placed at a more permanent memory location. You can name the temporary variable whatever you like, as long as it follows the naming standard of variable (e.g. no space, no number first)

```
drinks = ['coffee', 'coke', 'juice']

for option in drink:
    print(option)
```

Range in Loop

Sometimes we won't be iterating through a specific list, we'll just want to do a certain action multiple times.

#the use of range()

range(start, stop, step)

1.Looping over a range of numbers:

```
for num in range(5):
    print(num)
```

#Expected output: 0,1,2,3,4

2.Looping over a range with a specified start and stop:

```
for num in range(2,7):
    print(num)
```

#Expected output: 2,3,4,5,6

3.Looping over a range with a step value:

```
for num in range(0,10,2):
    print(num)
```

#Expected output: 0,2,4,6,8

Break/Continue Loop

The **break** and **continue** statements are used within loops (such as for and while loops) to control the flow of the loop execution.:

break

```
for num in range(10):
    if num ==5:
        break
    print(num)

#Result: 0,1,2,3,4
```

continue

```
for num in range(5):
    if num == 2:
        continue
    print(num)

#Result: 0,1,3,4
```

Looping Dictionary

- We loop a dictionary with the .items(), .keys() and .values() methods:

- Loop all keys with **.keys()**, for example:
  - for key in biggest_brands.keys():
    print(key)

- Loop all values with **.values()**, for example:
  - for value in biggest_brands.values():
    print(value)

- Loop all keys and values with **.items()**, for example:
  - for company, value in biggest_brands.items():
    print(f"{company} has a value of {value} billion dollars.")

List/Dictionary Comprehensions

List comprehensions and dictionary comprehensions are concise and expressive ways to create lists and dictionaries with loop, respectively.

List Comprehensions

new_list = [expression for item in iterable if condition]

Example:

```
squares_of_evens = [x ** 2 for x in range(10) if x % 2 == 0]
Print(squares_of_evens)
# Result: [0, 4, 16, 36, 64]
```

Dictionary Comprehensions

new_dict = {key_expression: value_expression for item in iterable if condition}

Example:

```
squares_dict = {x: x ** 2 for x in range(5)}
Print(squares_dict)
# Result: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

While Loops

The while loop performs a set of code until some condition is reached.

For example:

Suppose we have an operation: 1+2+3+4+.....
We want to know how many times we need to add, until it reaches a target

```
target = 12
current_number = 0
count = 0
while current_number < target:
    count += 1
    current_number += count
print(f"Need {count} times to reach {target}.")
```

Unit 6

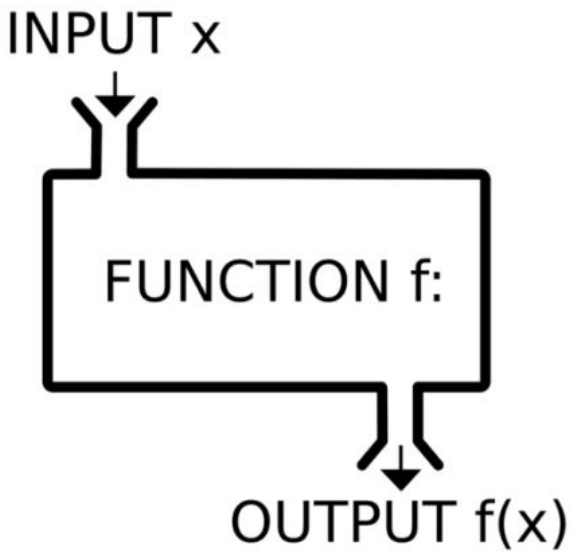
Functions

What will you learn

- Why we want to use function
- Position and keyword argument (from function definition point of view)
- Position and keyword argument (from function user’s point of view)
- How does the return mechanism works
- How does returning multiple values works
- Print is not the same as return

What is Function

- In Python, a function is a named set of instructions that you can use over and over again
- You give it some input, it does something with that input, and then it can give you back an output
- It's like having a mini-tool in your programming toolbox that you can use whenever you need to do a specific task



Write a function

- To write a function, you need:
  - The keyword def
  - The name of the function
  - Parameters delimited by commas
  - A colon
  - An indented block of code

```
def function(a, b):
    return a + b
```

Parameter and Argument

- **A parameter** is a variable listed in a function's definition that acts as a placeholder for the value that will be passed into the function when it's called.
- **An argument** is a value that is provided to a function when it's called, which corresponds to the parameter's placeholder and is used within the function's code.

```
# Function Definition
def function(a, b):
    return a + b
```

Parameter

```
# Function Call
function(a, b)
```

Argument

Parameter and Argument

- We can set **default value** to the parameter. In this example, param2 has a default value = 1:

  - ```
    def f1(param2=1):
        pass
    ```

- We can have more than 2 parameters. In this example, param3 has no default value and param4 has a default value = 1:

  - ```
    def f1(param3, param4=1):
        pass
    ```

Positional Arguments

Positional arguments refer to the values that you pass to a function based on their **position or order** in the function's parameter list.

```
def welcome_message(
        customer_name,
        welcome_1,
        welcome_2):

    print(f"Good morning {customer_name}")
    print(welcome_1)
    print(welcome_2)
```

```
welcome_message(
    "Jason",
    "Welcome to DANGER Bootcamp",
    "Enjoy your learning journey"
)
```

Positional Arguments

Keyword Arguments

The arguments as **keyword arguments**, where we explicitly refer to what each argument is assigned to in the function call.

```
def welcome_message(
        customer_name,
        welcome_1,
        welcome_2):

    print(f"Good morning {customer_name}")
    print(welcome_1)
    print(welcome_2)
```

```
welcome_message(
    customer_name = "Jason",
    welcome_1 = "Welcome to DANGER Bootcamp",
    welcome_2 = "Enjoy your learning journey"
)
```

Keyword Arguments

Multiple Parameters

- Function could be taken **more than one parameter** by using commas.

```
def welcome_message(
        customer_name,
        welcome_1,
        welcome_2):

    print(f"Good morning {customer_name}")
    print(welcome_1)
    print(welcome_2)
```

Three parameters on this function

- Positional arguments always go before keyword arguments
- All position arguments must be provided, but kwargs are optional

Returns

Functions can also return a value to the user so that this value can be modified or used later

```
def check_all_positive(num1, num2, num3):
    if num1 < 0:
        return False
    if num2 < 0:
        return False
    if num3 < 0:
        return False
    return True

is_all_positive = check_all_positive(1, -1, 0)

print("Is all positive:", is_all_positive)
```

Return Single value

```
def find_tallest(student_info):
    tallest_height = 0
    tallest_student = None
    for student, height in student_info.items():
        if height > tallest_height:
            tallest_height = height
            tallest_student = student
    return tallest_student, tallest_height

student_info = {"james": 170,"leo": 168,"timmy": 177}

name, measure = find_tallest(student_info)
```

Multiple Return Values

Unit 7

File operation

What will you learn

- How to read a file in Python
- How to write a file in Python

Read & Write a File

- With open(‘file name’) as f: Open the file

- f.read(): Read a file, for example:

```
with open('my_file.txt', 'r') as fp:
    content = fp.read()
```

- f.write(): Write a file, for example:

```
with open('my_file.txt', 'w') as fp:
    content = fp.write("hello")
```

Mode of file open

- **r:** Read a text file. fp.read() will return a string. Good for reading text files.
- **rb**: Read a binary file. fp.read() will return a byte-type. Good for reading non-text files.
- **w** : Overwrite a text file. fp.write() takes a string as input. Good for writing text.
- **wb** : Overwrite a binary file. fp.write() takes a byte-type as input. Good for writing non-text content.
- **a and ab** : Append instead of overwrite.